

Name: 徐靖甯

Date: 09/18

Topic: Recursion, Dictionaries

1. Today's Challenge

Core concept from today's OCW lecture: recursion and dictionaries.

I learned that recursion allows a function to call itself to solve a problem step by step, while dictionaries can be used to store and organize data using key-value pairs. These concepts can make some problems easier to solve by breaking them into smaller parts.

AI-generated coding challenge title: Mirror Equality

Challenge: Mirror Equality

Problem Statement

A string is called **mirror-equal** if the number of occurrences of the letter 'a' is exactly equal to the number of occurrences of the letter 'b'.

Write a function that determines whether a given string is mirror-equal.

Your solution must determine the answer by recursively processing the string.

Examples

- "abab" → True (2 'a's and 2 'b's)
- "aaabb" → False (3 'a's and 2 'b's)
- "ccddeeff" → True (0 'a's and 0 'b's)

Input

A single string `s`.

The string may contain lowercase letters, uppercase letters, digits, spaces, and punctuation.

Output

Return:

- True if the number of 'a' characters equals the number of 'b' characters
- False otherwise

Constraints

- $0 \leq \text{len}(s) \leq 200$
- Treat uppercase and lowercase letters as different characters.
- Only the characters 'a' and 'b' affect the result.
- All other characters should be ignored.

2. My Initial Approach — Before AI Help

My approach:

I planned to use recursion to check the string one character at a time. I would count 'a' as 1 and 'b' as -1, while ignoring other characters. After reaching the end of the string, I would check if the final count was 0.

Before asking AI for hints, briefly describe how you planned to solve the problem.

3. AI Tutor Help

Did you ask the AI Tutor for help?

☒ No — I solved it independently

☐ Yes — I received one or more hints

The most useful hint/question from AI was:

I did not ask the AI Tutor for help because I was able to solve the problem independently.

It helped me realize that:

It helped me realize that the base case is very important in recursion. Without a clear stopping condition, the function would keep calling itself and never finish. In this problem, reaching the end of the string is the condition that tells the recursion to stop.

4. My Revision

Did you change your approach or code after interacting with AI?

☒ No

☐ Yes

What did you change, and why?

I did not change my approach or code because I solved the problem independently.

5. Verification

My final program:

- Passed the provided examples
- Passed additional edge cases
- ☐ Still has unresolved problems

One edge case I tested:

Input: "A!B123"

Expected output: True

Actual output: True_

6. One-Minute Reflection

What idea from the OCW lecture did you transfer to this new problem?The idea of recursion

Instead of checking the entire string at once, I divided the problem into smaller steps by processing one character at a time. The function keeps calling itself until it reaches the end of the string.

One thing I understand better now:

I understand better how recursion uses a base case to stop the function. I also learned that I can keep useful information during the recursive process, such as the current difference between the number of 'a's and 'b's.

One thing I am still unsure about:

I am still a little unsure about how to decide when recursion is better than a loop. I understand how recursion works, but sometimes a loop seems simpler and easier to read. I think I need more practice with different types of recursion problems to understand when I should choose each method.